

Atividade prática: API de Gestão de Documentação Técnica e Certificados

1. Contexto

Você atua como Desenvolvedor Backend Júnior no departamento de Tecnologia da Informação de uma grande indústria multinacional do setor eletromecânico. Sua função principal no setor é projetar, desenvolver e manter microsserviços integrados que atendem às operações fabris, ao suporte pós-venda e à auditoria de qualidade dos equipamentos produzidos.

A empresa fabrica motores elétricos e geradores industriais de alta complexidade, operando sob rigorosas normas de compliance internacional e garantia de qualidade. Atualmente, a equipe de assistência técnica e auditoria enfrenta grande morosidade na consulta de dados de garantia: manuais de instalação, datasheets técnicos e certificados de conformidade estão dispersos em servidores de arquivos e planilhas legadas. Para resolver esse gargalo, o setor determinou a criação de um serviço centralizado que permita a consulta rápida de todos os documentos e parâmetros de um equipamento a partir de um único dado de entrada: o número de série (serial number) do produto.

2. Desafio

Seu desafio consiste em desenvolver um serviço backend em Spring Boot que centralize a gestão de documentação técnica e certificados por meio do número de série do produto, expondo uma API REST completa, persistindo dados relacionais com Spring Data JPA e disponibilizando a documentação técnica da aplicação via Swagger/OpenAPI e Javadoc.

3. Listagem dos Resultados e Entregas

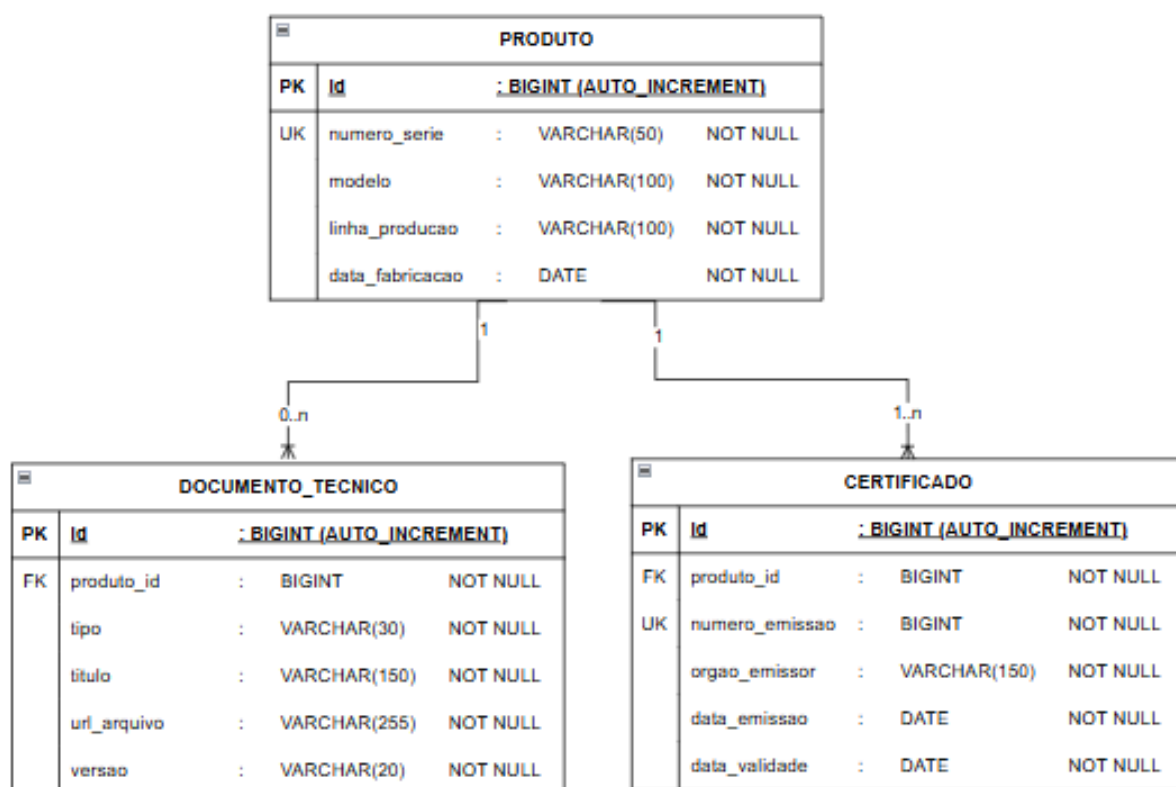
- **Código-fonte do projeto:** Repositório do projeto Spring Boot estruturado contendo a camada de modelo (entidades JPA), repositórios, serviços e controladores REST.
- **Mapeamento de banco de dados:** Mapeamento das entidades com seus respectivos relacionamentos e restrições.
- **Documentação de código:** Métodos das classes de serviço e controladores documentados utilizando o padrão Javadoc.
- **Interface da API (Swagger):** Endpoints mapeados e anotados para exibição e testes no Swagger UI (/swagger-ui.html).

4. Critérios de Avaliação

- **Spring JPA** (Anotações de Mapeamento das Entidades e dos Relacionamentos) [60% da nota]
 - O estudante mapeou as classes de modelo utilizando as anotações `@Entity`, `@Table`, `@Id` e `@GeneratedValue` conforme as restrições e chaves primárias do modelo relacional?
 - O estudante configurou os relacionamentos `@OneToMany`, `@ManyToOne` e `@OneToOne` utilizando os atributos `mappedBy`, `joinColumns` e as estratégias de busca (`FetchType`) estabelecidas no projeto?
 - O estudante declarou as restrições de integridade e unicidade dos campos das entidades utilizando as propriedades `nullable`, `unique` e `length` na anotação `@Column`?

- O esquema de banco de dados gerado pelo Hibernate reflete as tabelas, chaves estrangeiras e índices conforme as regras de associação entre produtos, documentos e certificados?
- A classe de modelo Produto contém o atributo do número de série anotado com restrição de unicidade e indexação para consultas?
- Os repositórios Spring Data JPA estendem JpaRepository disponibilizando métodos de consulta customizados por número de série sem a necessidade de SQL nativo?
- **API Rest** (Métodos HTTP, Headers, Códigos de Status) [30% da nota]
 - O estudante estruturou os controladores REST utilizando as anotações @RestController e @RequestMapping associando as rotas aos verbos HTTP GET, POST, PUT e DELETE?
 - O estudante tratou os parâmetros de requisição utilizando @PathVariable, @RequestParam e @RequestBody conforme os padrões RESTful?
 - O estudante configurou a manipulação de exceções da aplicação retornando instâncias de ResponseEntity contendo os códigos de status HTTP específicos para cada situação?
 - O endpoint de consulta por número de série retorna o código de status HTTP 200 OK com o payload JSON da documentação quando o número de série existe no banco de dados?
 - O endpoint de inserção de documentos retorna o código de status HTTP 201 Created contendo o header Location com a URI do recurso criado?
 - O endpoint de busca retorna o código de status HTTP 404 Not Found quando o número de série informado não possui registros no sistema?
- **Documentação** com Swagger e Javadoc [10% da nota]
 - O estudante documentou os métodos das classes da camada de controle e serviço utilizando blocos Javadoc com as tags @param, @return e @throws?
 - O estudante adicionou as dependências do Springdoc OpenAPI no arquivo de build do projeto (pom.xml ou build.gradle) para geração automática da especificação Swagger?
 - O estudante anotou os endpoints e DTOs com @Operation, @ApiResponse e @Schema detalhando as respostas e descrições dos atributos no Swagger?
 - O arquivo Javadoc gerado ou visualizável no código apresenta a descrição textual dos parâmetros de entrada e retornos de cada método exposto?
 - A interface Swagger UI acessível via navegador exibe a lista completa de endpoints com suas respectivas descrições e códigos de retorno HTTP atestando a capacidade de testes?
 - O esquema JSON do Swagger documenta os atributos das entidades e DTOs especificando tipos de dados e obrigatoriedade de preenchimento?

ANEXO I - DER (DIAGRAMA DE ENTIDADE E RELACIONAMENTO)



ANEXO II - ESPECIFICAÇÃO DAS ENTIDADE E REGRAS DE NEGÓCIO

Produto

- Propósito: Representa o equipamento físico gerado na fábrica (motores, geradores).
- numero_serie (UK - Unique Key): O identificador de negócio exclusivo do equipamento. Deve ter restrição de unicidade (unique = true) e índice para consultas rápidas.

Documento técnico

- Propósito: Guarda os manuais de instrução, esquemas elétricos e datasheets.
- Relacionamento com Produto: 1 para N (@ManyToOne em DocumentoTecnico, @OneToMany em Produto). Um produto pode ter múltiplos documentos vinculados (ex: 1 Manual do Usuário + 1 Datasheet + 1 Guia de Manutenção).
- Atributo tipo: Define o tipo de documento (MANUAL, DATASHEET, DIAGRAMA_ELETRICO).

Certificado

- Propósito: Armazena os certificados de conformidade e qualidade (ISO, ATEX, Inmetro, CE).
- Relacionamento com Produto: 1 para N (@ManyToOne em Certificado, @OneToMany em Produto). Um produto pode possuir mais de um certificado emitido por diferentes órgãos reguladores.
- numero_emissao (UK): Identificador do documento legal emitido pelo órgão certificador.

ANEXO III - SCRIPT DDL (GERAÇÃO DE TABELAS EM SQL)

```
CREATE TABLE produto (  
    id BIGINT AUTO_INCREMENT PRIMARY KEY,  
    numero_serie VARCHAR(50) NOT NULL UNIQUE,  
    modelo VARCHAR(100) NOT NULL,  
    linha_producao VARCHAR(100) NOT NULL,  
    data_fabricacao DATE NOT NULL  
);  
  
CREATE TABLE documento_tecnico (  
    id BIGINT AUTO_INCREMENT PRIMARY KEY,  
    produto_id BIGINT NOT NULL,  
    tipo VARCHAR(30) NOT NULL,  
    titulo VARCHAR(150) NOT NULL,  
    url_arquivo VARCHAR(255) NOT NULL,  
    versao VARCHAR(20) NOT NULL,  
    CONSTRAINT fk_doc_produto FOREIGN KEY (produto_id) REFERENCES produto(id) ON DELETE  
CASCADE  
);  
  
CREATE TABLE certificado (  
    id BIGINT AUTO_INCREMENT PRIMARY KEY,  
    produto_id BIGINT NOT NULL,  
    numero_emissao VARCHAR(50) NOT NULL UNIQUE,  
    orgao_emissor VARCHAR(100) NOT NULL,  
    data_emissao DATE NOT NULL,  
    data_validade DATE,  
    CONSTRAINT fk_cert_produto FOREIGN KEY (produto_id) REFERENCES produto(id) ON DELETE  
CASCADE  
);
```

ANEXO IV - GUIA DE MAPEAMENTO JPA E CONFIGURAÇÃO

Configuração do application.properties (Banco em Memória H2)

```
spring.datasource.url=jdbc:h2:mem:gestaodb
spring.datasource.driverClassName=org.h2.Driver
spring.datasource.username=sa
spring.datasource.password=
spring.h2.console.enabled=true
spring.jpa.database-platform=org.hibernate.dialect.H2Dialect
spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true
```

Estrutura Geral das Entidades

Todas as classes de modelo devem ser anotadas como entidades JPA e mapeadas para suas respectivas tabelas no banco de dados.

- **@Entity**: Indica ao Spring/Hibernate que a classe representa uma tabela do banco de dados relacional.
- **@Table(name = "nome_tabela")**: Define explicitamente o nome da tabela no banco de dados em formato snake_case (padrão SQL).
- **Encapsulamento e Construtores**: Todas as entidades devem possuir um construtor sem argumentos (protected ou public, exigido pelo JPA) e os métodos getters e setters.

Mapeamento de Chaves Primárias, Atributos e Unicidade

O identificador único de negócio (numeroSerie) exige configurações específicas para garantir a performance e a integridade do banco.

- **Identificador Primário (@Id e @GeneratedValue)**:
 - Sempre utilize o tipo envelopado (Long ou BigInteger) para a PK, evitando o tipo primitivo long (que inicia como 0 em vez de null).
 - Use **@GeneratedValue(strategy = GenerationType.IDENTITY)** para delegar o auto-incremento ao próprio banco de dados (coluna AUTO_INCREMENT ou SERIAL).
- **Restrições de Colunas (@Column)**:
 - Para o campo numeroSerie: **@Column(name = "numero_serie", nullable = false, unique = true, length = 50)**
 - **nullable = false**: Aplica a restrição NOT NULL na tabela.
 - **unique = true**: Cria o índice de unicidade no banco de dados, impedindo produtos duplicados.
 - **length = 50**: Limita o tamanho do campo no banco, otimizando o uso de memória.

Relacionamentos Bidirecionais (@OneToMany / @ManyToOne)

O relacionamento entre Produto (Pai) e DocumentoTecnico/Certificado (Filhos) é bidirecional. Nesses casos, existe um lado "dono" do relacionamento e um lado "espelho".

- **Entidades Filhas (DocumentoTecnico e Certificado) — O Lado "Dono"**:
 - São elas que contêm a chave estrangeira (Foreign Key) no banco de dados.

- Usa-se `@ManyToOne(fetch = FetchType.LAZY)`. O `FetchType.LAZY` é fundamental para evitar que a aplicação carregue todos os documentos do banco de dados desnecessariamente ao buscar apenas o produto.
- Usa-se `@JoinColumn(name = "produto_id", nullable = false)` para indicar o nome exato da coluna da FK.
- Entidade Pai (Produto) — O Lado Mapeado/Espelho:
 - Contém as coleções (`Set<DocumentoTecnico>` e `Set<Certificado>`).
 - Usa-se `@OneToMany(mappedBy = "produto", cascade = CascadeType.ALL, orphanRemoval = true)`.
 - `mappedBy = "produto"`: Aponta para o nome do atributo Java declarado na classe filha (não o nome da coluna do banco).
 - `cascade = CascadeType.ALL`: Garante que ao salvar, atualizar ou excluir um Produto, seus documentos e certificados associados sejam processados automaticamente.
 - `orphanRemoval = true`: Remove do banco de dados qualquer documento que for desvinculado da lista do produto.

Prevenção de Erros Comuns no Teste da AP

- Loop Infinito na Serialização JSON (Recursão Bidirecional):
 - Como Produto aponta para DocumentoTecnico e DocumentoTecnico aponta para Produto, a conversão do objeto para JSON gerará um erro de estouro de memória (`StackOverflowError`).
 - **Solução recomendada:** Utilizar a anotação `@JsonIgnoreProperties("produto")` ou `@JsonBackReference` no atributo produto dentro das entidades filhas, ou atuar com DTOs (Data Transfer Objects) nas respostas dos Controllers.
- Inicialização de Coleções:
 - Inicialize sempre as coleções na entidade pai para evitar exceções de ponteiro nulo (`NullPointerException`):

```
private Set<DocumentoTecnico> documentos = new HashSet<>();
```

ANEXO V - GUIA DE APROFUNDAMENTO PARA API REST (CONTRATO DOS ENDPOINTS)

Estrutura e Rotas dos Controladores (@RestController)

A camada de controle expõe as funcionalidades da aplicação para o cliente REST, tratando dados de entrada e formatando as respostas de acordo com o padrão arquitetural HTTP.

- **@RestController**: Anotação composta por **@Controller** e **@ResponseBody**. Indica ao Spring que os retornos dos métodos devem ser serializados diretamente no corpo da resposta HTTP (JSON por padrão).
- **@RequestMapping("/api/v1/produtos")**: Define a rota-base da API. O versionamento na URL (/v1) é uma boa prática para controle de compatibilidade.

Mapeamento de Métodos e Semântica HTTP

A API deve respeitar a semântica dos verbos HTTP para representar as operações CRUD no recurso:

- **@GetMapping (Consulta)**:
 - **GET /api/v1/produtos/{numeroSerie}**: Busca um produto e sua documentação associada.
 - **Sintaxe do parâmetro**: Usa-se **@PathVariable String numeroSerie** para extrair o valor informado diretamente na URL.
- **@PostMapping (Criação)**:
 - **POST /api/v1/produtos**: Cadastra um novo produto.
 - **POST /api/v1/produtos/{numeroSerie}/documentos**: Vincula um novo documento técnico a um produto existente.
 - **Sintaxe do parâmetro**: Usa-se **@RequestBody @Valid ProdutoDTO dto** para converter o JSON enviado na requisição em um objeto Java e aplicar validações.
- **@PutMapping / @PatchMapping (Atualização)**:
 - **PUT /api/v1/produtos/{numeroSerie}**: Atualiza os dados cadastrais do produto.
- **@DeleteMapping (Remoção)**:
 - **DELETE /api/v1/produtos/{numeroSerie}/documentos/{idDocumento}**: Remove um documento do sistema.

Manipulação de Headers e Códigos de Status HTTP (ResponseEntity)

Nunca retorne apenas a entidade direta no método do controlador. Utilize sempre a classe **ResponseEntity<T>** para controlar de forma explícita o Status Code, os Headers e o Payload.

- **200 OK**: Para consultas (GET) ou atualizações (PUT) bem-sucedidas.

```
return ResponseEntity.ok(produtoDTO);
```

- **201 Created**: Para criação de recursos (POST). Exige a inclusão do header Location com a URI de acesso ao novo recurso.

```
URI uri = ServletUriComponentsBuilder.fromCurrentRequest()
```

```
.path("/{id}")  
.buildAndExpand(novoCertificado.getId())  
.toUri();  
return ResponseEntity.created(uri).body(novoCertificadoDTO);
```

- 204 No Content: Para exclusões efetuadas com sucesso (DELETE) onde não há corpo de resposta.

```
return ResponseEntity.noContent().build();
```

- 404 Not Found: Retornado quando o numeroSerie pesquisado não existir no banco de dados.
- 400 Bad Request: Retornado quando a requisição contiver dados inválidos ou falhas de validação sintática.
- 409 Conflict: Retornado quando houver tentativa de cadastrar um numeroSerie que já existe no sistema.

Tratamento Global de Exceções (@RestControllerAdvice)

Para evitar o uso excessivo de blocos try-catch dentro dos métodos dos controladores, centralize o tratamento de erros da aplicação utilizando um manipulador global:

- @RestControllerAdvice: Captura exceções lançadas em qualquer camada da aplicação (Service/Repository) e as converte em respostas HTTP padronizadas.
- @ExceptionHandler(RecursoNaoEncontradoException.class): Mapeia exceções customizadas para um status HTTP específico (ex: converte uma exceção de produto não encontrado em um status 404 Not Found).

Prevenção de Erros Comuns na API REST

- Não expor entidades JPA diretamente na API: Utilize objetos DTO (Data Transfer Objects) para receber e retornar dados nos controladores, prevenindo o vazamento de detalhes do banco de dados e loops de serialização.
- Atenção aos Verbos HTTP: Não utilize GET para alterar dados no servidor nem POST para consultas puras.
- Tratamento de Recurso Ausente: Ao consultar por numeroSerie e não encontrar registros, a API deve obrigatoriamente retornar 404 Not Found em vez de um corpo vazio com status 200 OK.

ANEXO VI - GUIA DE APROFUNDAMENTO DA DOCUMENTAÇÃO COM SWAGGER/OPENAPI E JAVADOC

Documentação do Código-Fonte com Javadoc

O Javadoc documenta a arquitetura, métodos e regras de negócio diretamente no código Java, facilitando a manutenção da aplicação por outros desenvolvedores da equipe.

- Estrutura de Bloco: Deve ser iniciado por `/**` e finalizado por `*/`.
- Aplicações Recomendadas:
 - Classes de Serviço (`@Service`): Documentação das regras de negócio (ex: regras de validação de número de série).
 - Métodos do Controller (`@RestController`): Descrição do objetivo operacional do endpoint.
- Tags Obrigatórias a Utilizar:
 - `@param`: Descreve o significado e as restrições dos parâmetros de entrada recebidos pelo método.
 - `@return`: Explica o tipo de dado e a estrutura retornada pelo método após a execução.
 - `@throws`: Documenta as exceções lançadas pela aplicação em cenários de erro ou regras de negócio violadas.
- Exemplo de Aplicação (Serviço/Controller):

```
/**
 * Busca os documentos técnicos e certificados associados a um equipamento.
 *
 * @param numeroSerie Número de série único gravado no equipamento.
 * @return DTO contendo os dados do produto, lista de manuais e certificados.
 * @throws RecursoNaoEncontradoException Caso o número de série não seja localizado
 no banco de dados.
 */
public ProdutoDTO buscarPorNumeroSerie(String numeroSerie) { ... }
```

Configuração e Dependências do Swagger (Springdoc OpenAPI)

O Swagger gera uma interface gráfica interativa (`/swagger-ui.html`) permitindo que clientes e equipes de testes consumam e experimentem a API diretamente no navegador.

- Dependência no pom.xml (Maven): O projeto deve incluir a biblioteca oficial do Springdoc OpenAPI para Spring Boot 3:

```
<dependency>
  <groupId>org.springdoc</groupId>
  <artifactId>springdoc-openapi-starter-webmvc-ui</artifactId>
  <version>2.3.0</version>
</dependency>
```

- Classe de Configuração (`@Configuration`): Utilize a anotação `@OpenAPIDefinition` para customizar as informações do cabeçalho da documentação:

```
@Configuration
@OpenAPIDefinition(
```

```

        info = @Info(
            title = "API de Gestão de Documentação Técnica e Certificados",
            version = "1.0",
            description = "Serviço centralizado para consulta de manuais e certificados por
número de série."
        )
    )
}

public class SwaggerConfig {}

```

Anotações do Swagger nos Controllers e DTOs

Para enriquecer a documentação exposta no Swagger UI, os métodos do controller e os atributos dos DTOs devem ser anotados com metadados descritivos.

- Anotações no Controller (@RestController):
 - @Tag(name = "...", description = "..."): Agrupa e categoriza os endpoints por recurso na interface do Swagger UI.
 - @Operation(summary = "...", description = "..."): Descreve resumidamente a ação executada pelo endpoint e seus detalhes operacionais.
 - @ApiResponses e @ApiResponse: Mapeia explicitamente todos os códigos de status HTTP possíveis que o endpoint pode retornar.
 - Exemplo no Controller:

```

@Operation(summary = "Buscar documentação por número de série", description = "Retorna os
manuais, datasheets e certificados vinculados ao equipamento.")
@ApiResponses(value = {
    @ApiResponse(responseCode = "200", description = "Equipamento localizado com sucesso"),
    @ApiResponse(responseCode = "404", description = "Número de série não cadastrado no
sistema")
})
@GetMapping("/{numeroSerie}")
public ResponseEntity<ProdutoDTO> buscarPorNumeroSerie(@PathVariable String
numeroSerie) { ... }

```

Anotações nos DTOs/Esquemas (@Schema):

Documenta a estrutura das requisições e respostas JSON, especificando exemplos práticos de uso e a obrigatoriedade dos campos.

Exemplo DTO:

```

public class ProdutoDTO {
    @Schema(description = "Número de série gravado na placa do equipamento", example =
"MOT-2024-8890", requiredMode = Schema.RequiredMode.REQUIRED)
    private String numeroSerie;

    @Schema(description = "Modelo comercial do produto", example = "Motor Trifásico W22")
    private String modelo;
    // Getters e Setters
}

```

Prevenção de Erros Comuns na Documentação

- Esquecer de Documentar Respostas de Erro: Não limite as @ApiResponse apenas ao código 200 OK. Documente obrigatoriamente os retornos de erro (400 Bad Request, 404 Not Found, 409 Conflict).
- Javadoc Incompleto: Blocos Javadoc sem as tags @param ou @return devidamente preenchidas serão desconsiderados na avaliação do critério C3.
- Exposição de Rotas Internas: Evite expor no Swagger rotas internas ou do manipulador de exceções. Documente apenas os endpoints de negócio sob /api/v1/produtos.

ANEXO VII - CARGA INICIAL DE DADOS DE TESTE (src/main/resources/data.sql)

Crie o arquivo data.sql no diretório src/main/resources para popular o banco de dados automaticamente ao iniciar a aplicação.

Dicas de Cenários de Teste:

Com essa massa de dados, você poderão validar a API nos seguintes cenários:

1. **Consulta bem-sucedida com múltiplos relacionamentos (GET /api/v1/produtos/MOT-2026-9901):** Retorna o produto com 2 documentos e 2 certificados.
2. **Consulta de produto que possui documentos, mas nenhum certificado (GET /api/v1/produtos/MOT-2026-9903):** Válido para testar como o DTO lida com coleções vazias sem quebrar o retorno JSON.
3. **Consulta de produto inexistente (GET /api/v1/produtos/NULO-0000):** Garante o retorno correto do código 404 Not Found.
4. **Validação de Unicidade de Número de Série (POST /api/v1/produtos com MOT-2026-9901):** Testa a captura da exceção de duplicidade no banco e retorno do status 409 Conflict ou 400 Bad Request.

```
--
=====
-- CARGA DE DADOS DE TESTE: PRODUTOS (EQUIPAMENTOS)
--
=====
INSERT INTO produto (id, numero_serie, modelo, linha_producao, data_fabricacao)
VALUES (1, 'MOT-2026-9901', 'Motor Trifásico W22 50HP', 'Linha Alta Tensão 02', '2026-02-15');

INSERT INTO produto (id, numero_serie, modelo, linha_producao, data_fabricacao)
VALUES (2, 'MOT-2026-9902', 'Motor Antideflagrante W22Xdb', 'Linha Ambientes Explosivos',
'2026-01-20');

INSERT INTO produto (id, numero_serie, modelo, linha_producao, data_fabricacao)
VALUES (3, 'GER-2026-5001', 'Gerador Industrial Diesel GTA-500kW', 'Linha de Geradores
Pesados', '2025-11-10');

INSERT INTO produto (id, numero_serie, modelo, linha_producao, data_fabricacao)
VALUES (4, 'SRV-2026-1040', 'Servomotor de Alta Precisão SWA-40', 'Linha de Automação
Robótica', '2026-03-01');

INSERT INTO produto (id, numero_serie, modelo, linha_producao, data_fabricacao)
VALUES (5, 'MOT-2026-9903', 'Motor Monofásico NEMA Premium 10HP', 'Linha Comercial Leve',
'2026-02-28');

--
=====
-- CARGA DE DADOS DE TESTE: DOCUMENTOS TÉCNICOS
```

```
--
=====
-- Documentos do Produto 1 (MOT-2026-9901)
INSERT INTO documento_tecnico (id, produto_id, tipo, titulo, url_arquivo, versao)
VALUES (1, 1, 'MANUAL', 'Manual de Instalação e Operação W22',
'https://docs.fabrica.com/manuais/w22.pdf', 'v2.1');

INSERT INTO documento_tecnico (id, produto_id, tipo, titulo, url_arquivo, versao)
VALUES (2, 1, 'DATASHEET', 'Folha de Dados Técnicos Curva de Torque W22',
'https://docs.fabrica.com/datasheets/w22-50hp.pdf', 'v1.0');

-- Documentos do Produto 2 (MOT-2026-9902)
INSERT INTO documento_tecnico (id, produto_id, tipo, titulo, url_arquivo, versao)
VALUES (3, 2, 'MANUAL', 'Guia de Segurança em Áreas Classificadas ATEX',
'https://docs.fabrica.com/manuais/atex-w22xdb.pdf', 'v3.0');

INSERT INTO documento_tecnico (id, produto_id, tipo, titulo, url_arquivo, versao)
VALUES (4, 2, 'DIAGRAMA_ELETRICO', 'Esquema de Ligação da Caixa de Bornes Ex',
'https://docs.fabrica.com/esquemas/w22xdb-bornes.pdf', 'v1.2');

-- Documentos do Produto 3 (GER-2026-5001)
INSERT INTO documento_tecnico (id, produto_id, tipo, titulo, url_arquivo, versao)
VALUES (5, 3, 'MANUAL', 'Manual de Manutenção Preventiva do Alternador',
'https://docs.fabrica.com/manuais/gta500-maint.pdf', 'v1.5');

INSERT INTO documento_tecnico (id, produto_id, tipo, titulo, url_arquivo, versao)
VALUES (6, 3, 'DATASHEET', 'Especificação de Consumo de Combustível e Potência',
'https://docs.fabrica.com/datasheets/gta500.pdf', 'v2.0');

-- Documentos do Produto 4 (SRV-2026-1040)
INSERT INTO documento_tecnico (id, produto_id, tipo, titulo, url_arquivo, versao)
VALUES (7, 4, 'DIAGRAMA_ELETRICO', 'Pinagem de Controle Encoder Resolver',
'https://docs.fabrica.com/esquemas/swa40-pinout.pdf', 'v1.0');

--
=====
-- CARGA DE DADOS DE TESTE: CERTIFICADOS
--
=====
-- Certificados do Produto 1 (MOT-2026-9901)
INSERT INTO certificado (id, produto_id, numero_emissao, orgao_emissor, data_emissao,
data_validade)
VALUES (1, 1, 'CERT-ISO-8821', 'Bureau Veritas', '2026-01-10', '2029-01-10');

INSERT INTO certificado (id, produto_id, numero_emissao, orgao_emissor, data_emissao,
data_validade)
VALUES (2, 1, 'CERT-INMETRO-2026-04', 'TÜV Rheinland', '2026-02-01', '2028-02-01');

-- Certificados do Produto 2 (MOT-2026-9902)
INSERT INTO certificado (id, produto_id, numero_emissao, orgao_emissor, data_emissao,
data_validade)
```

```
VALUES (3, 2, 'CERT-ATEX-EX-9902', 'DNV GL Europe', '2026-01-05', '2031-01-05');
```

```
INSERT INTO certificado (id, produto_id, numero_emissao, orgao_emissor, data_emissao,  
data_validade)
```

```
VALUES (4, 2, 'CERT-IECEX-2026-88', 'DEKRA Certification', '2026-01-15', '2031-01-15');
```

```
-- Certificados do Produto 3 (GER-2026-5001)
```

```
INSERT INTO certificado (id, produto_id, numero_emissao, orgao_emissor, data_emissao,  
data_validade)
```

```
VALUES (5, 3, 'CERT-CE-MARK-2025', 'SGS Group', '2025-11-20', '2028-11-20');
```

```
-- Certificados do Produto 4 (SRV-2026-1040)
```

```
INSERT INTO certificado (id, produto_id, numero_emissao, orgao_emissor, data_emissao,  
data_validade)
```

```
VALUES (6, 4, 'CERT-UL-ROBOTICS-01', 'Underwriters Laboratories (UL)', '2026-02-10',  
'2030-02-10');
```